

ENSC 254 Milestone 4 Report

Introduction

For milestone 4, our simulator was extended with four new features that improve either processor performance or simulator usability. The implemented features consist of an instruction cache, two branch prediction methods, and a terminal based user interface. The primary purpose of these enhancements is to reduce the amount of memory stalls, improve branch execution efficiency and simplify testing of different simulator configurations.

Feature 1 (Part 1 in Code)- Instruction Cache

Description

An instruction cache was implemented to reduce the time required to fetch instructions from memory. In a pipeline processor every instruction must first pass through the IF (instruction Fetch) stage before it can be decoded and executed. Accessing the main memory of every single instruction introduces unnecessary latency especially when the processor repeatedly executes the same sequence of instructions inside loops.

To address this, an independent instruction cache was added to the fetch stage so before an instruction is read from memory, the simulator checks whether the instruction is already stored in the cache. If the instruction is present (is a cache hit) it can be supplied immediately, otherwise, when there is a cache miss, the instruction is fetched from memory and inserted into the cache for future accesses.

The cache was integrated by passing an instruction cache object into the fetch stage:

`stage_fetch(..., Cache *icache_p)`, and additional counters were also added to record hits, misses, and evictions throughout the execution.

Impact

The instruction cache improves processor performance by exploiting temporal locality which is that recently executed instructions are likely to be executed again. Programs containing loops benefit the most because the same instructions are fetched repeatedly after the first iteration. During testing, the instruction cache produced significantly more hits than misses, demonstrating that repeated instruction fetches were successfully served from the cache instead of main memory. The simulator reports the following to evaluate the performance of the cache:

- Instruction cache accesses
- Instruction cache hits

- Instruction cache misses
- Cache evictions

Our results matched expectations, as instruction caching is widely used in modern processors to reduce memory access latency and improve execution efficiency.

Feature 2 (Part 2 in Code) - One Branch Bit Predictor

Description

Our second feature is a one bit branch predictor to reduce the performance penalty associated with branch instructions. Without branch prediction, the processor must wait until a branch instruction is resolved before knowing which instruction should be fetched next, resulting in pipeline stalls or flushed instructions.

The predictor will store the outcome of the most recent execution of each branch instruction. If the previous execution was taken the predictor assumes the next execution will also be taken. Likewise, if the previous execution was not taken it predicts the same outcome.

Each predictor entry stores a valid bit and the previous branch result as implemented in the code:

```
typedef struct {
    bool valid;
    bool last_taken;
} one_bit_predictor_entry_t;
```

After the branch is resolved, the stored prediction is updated so future executions use the latest branch behaviour.

Impact

The one-bit predictor improves the efficiency of the pipeline when the branch behaviour is consistent due to the fact that the processor is able to correctly predict the next instruction path before the branch has completed execution.

This implementation works well for branches that are usually taken or not taken but when the branch outcomes alternate frequently, its accuracy decreases because a single incorrect prediction immediately changes the stored prediction.

To evaluate performance, the simulator records:

- Total branch predictions
- Correct predictions
- Incorrect predictions

- Overall prediction accuracy

These statistics provide a quantitative measure of predictor performance and allow comparison with more advanced prediction methods.

Feature 3 (Part 3 in Code) - Two bit Saturating Branch Predictor

Description:

Our third feature we added was a two bit saturating branch predictor and was added as an improvement over the one bit predictor. Rather than changing its prediction after an incorrect outcome, the predictor uses a four state finite state machine consisting of strongly taken, weakly taken, weakly not taken, and strongly not taken. The predictor only changes from predicting “taken” to “not taken” or vice versa after there are two consecutive incorrect outcomes. This prevents isolated branch anomalies from immediately changing the prediction.

Impact:

The two bit predictor provides more stable predictions than the one bit predictor because it is less sensitive to occasional changes in branch behaviour. Programs containing loops typically execute the same branch many times before exiting, making the two bit predictor very effective. Testing showed that the predictor successfully tracked branch history while producing prediction statistics that can be compared with the one bit implementation. Although this predictor does require slightly more storage per entry, the increase is very small compared to the improvement in prediction stability.

Overall, the observed behavior matched expectations from computer architecture theory where two bit predictors are commonly used because they reduce unnecessary prediction changes while maintaining a simple implementation.

Feature 4 (Part 4 in Code)- Terminal Graphical User Interface (GUI)

Description:

A GUI was developed to simplify interaction with the simulator. Prior to this addition, changing the simulator configurations required manually entering build commands and command line arguments, making it difficult to test and probability of error. The interface was implemented as a bash script that presents as an interactive menu from which users can build the simulator, select branch predictor modes, execute test cases and configure simulator options.

Impact:

The terminal GUI improves the usability of the simulator by providing a centralized interface for configuring and executing experiments. Instead of memorizing long build commands, users can quickly select options through the menu system.

The GUI itself does not directly improve processor performance but it significantly improves the efficiency of testing and debugging by reducing setup time and minimizing user errors in the command line. This makes it easier to evaluate different simulator configurations and repeat experiments consistently.

TEST CASES:

Before running the test cases run these in case of permission errors

```
chmod +x test_simulator_ms4.sh
chmod +x test_simulator_ms4_branch_predictor.sh
chmod +x test_simulator_ms4_2bit.sh
chmod +x terminal_gui.sh
chmod +x test_all_ms4.sh
```

Test 1- Basic Instruction Cache test

Purpose:

Verify the instruction cache is correctly integrated into the IF stage and records cache activity during the execution of the program.

Expected Result:

The simulator should execute successfully and display cache statistics, like accesses, hits, and misses,

How to test:

```
./test_simulator_ms4.sh
```

Test 2 - Instruction Cache Eviction Test

Purpose:

Verify the instruction replaces cache blocks when capacity is exceeded.

Expected Result:

The simulator should report cache statistics, including one or more cache evictions, indicating that the replacement policy is functioning correctly.

How to test:

```
./test_simulator_ms4_icache.sh
```

Test 3 - One Bit Branch Predictor Test

Purpose:

Verify the one bit branch predictor correctly stores and updates the previous outcome of branch instructions.

Expected Result:

The simulator should display branch prediction statistics including the total number of predictions, correct predictions, incorrect predictions, and overall prediction accuracy.

How to test:

```
make clean  
make BP_MODE=1  
./test_simulator_ms4_branch_predictor.sh
```

Test 4 - Two Bit Branch Predictor Test**Purpose:**

Verify that the two bit saturating branch predictor correctly updates its prediction states and generates branch prediction statistics

Expected Result:

The simulator should indicate that the two bit predictor is active and display prediction statistics, including total predictions, incorrect predictions, and prediction accuracy.

How to Test:

```
make clean  
make BP_MODE=2  
./test_simulator_ms4_2bit.sh
```

Test 5 - Terminal GUI Test**Purpose:**

Verify that the terminal GUI launches correctly and allows the user to interact with the simulator

Expected Result:

The menu should appear without errors, allowing the user to navigate menu options, configure simulator settings, and execute available commands.

How to Test:

```
./terminal_gui.sh
```

Results and Discussion:

All four implemented features were successfully integrated into the RISC-V pipeline simulator and passed their respective test cases. The instruction cache, branch predictors, and terminal GUI all functioned as intended without affecting the operation of the existing simulator.

The instruction cache behaved as expected during testing. The basic instruction cache test confirmed that instruction access, cache hits, and cache misses were correctly recorded during execution. The larger cache eviction test further verified that the cache replacement worked as intended when the cache became full because of the number of evictions. As anticipated, programs containing repeated instruction execution produced a higher number of cache hits than misses because frequently executed instructions remain in the cache. These results show that the instruction cache successfully exploited temporal locality to reduce simulated instruction fetch latency.

Both branch predictors correctly tracked branch outcomes and generated prediction statistics. The one bit branch predictor successfully updated its prediction after each branch execution and achieved good prediction accuracy for the provided test program (75%). Although the two bit branch predictor is generally expected to provide more stable predictions, it produced a lower prediction accuracy than the one bit predictor in our testing (57.1%). This result is because the branch pattern used in the test program changed frequently. Since the two bit predictor requires two consecutive incorrect outcomes before changing its prediction, it adapts more slowly to rapidly changing branch behaviour. In contrast, the one bit predictor changes its prediction after a single incorrect outcome, allowing it to better match the specific branch pattern used during testing. Therefore, the observed results are consistent with the expected characteristics of both prediction algorithms rather than indicating an implementation error.

The terminal GUI also performed as expected. The interactive interface allowed simulator options and branch prediction modes to be selected without manually entering multiple command line arguments avoiding any user errors and simplifying testing.

Overall, the implemented features met the objectives of Milestone 4. The instruction cache improved simulated memory performance by reducing repeated instruction fetches, the branch predictors demonstrated realistic branch prediction behaviour, and the terminal GUI improved the usability of the simulator. The testing and results show that the implementations were successfully integrated into the existing pipeline simulator.